

Homework Exercises 9

Dr. Sarvar Abdullaev
s.abdullaev@inha.uz

November 19, 2022

After completing all lab exercises below, zip all your solution files into a single archive with your student ID (e.g. u210023.zip) and upload it to eClass before the official deadline.

1 Resolution Method

Predicate logic, unlike *propositional logic* which we discussed in lecture, allows us to make statements including variables. For example, in propositional logic P may represent some statement such as "Socrate is a man". Whereas, in predicate logic, $P(X)$ will refer to a statement "X is a man", and if "a is Socrate", then $P(a)$ will mean "Socrate is a man". We can also define general *rules* in predicate logic, such $P(X) \rightarrow Q(X)$, meaning something like "If X is a man, X is mortal". This will allow us to deduce new facts using Aristotelian syllogism:

$$\frac{P(X) \rightarrow Q(X), P(a)}{\therefore Q(a)}$$

Therefore **Socrate is mortal**.

Using predicate logic and resolution method, we can reach interesting conclusions. Suppose we have a following knowledge:

- Sarvar likes all kind of food.
- Apple and pear are food.
- Anything anyone eats and does not die is food.
- Alisher eats peanuts and still alive.
- Jamol eats everything that Alisher eats.

Let's convert our knowledge into predicate format as follows:

Predicate Logic	English
$food(X) \rightarrow likes(sarvar, X)$	If X is food, then Sarvar likes it.
$food(apple) \wedge food(pear)$	Apple and pear are food.
$eats(X, Y) \wedge \neg dead(X) \rightarrow food(Y)$	If X eats Y and X is not dead, then Y is food.
$eats(alisher, peanuts) \wedge alive(alisher)$	Alisher eats peanuts and he is alive.
$eats(alisher, X) \rightarrow eats(jamol, X)$	If Alisher eats X, Jamol eats X.
$alive(X) \rightarrow \neg dead(X)$	If X is alive, X is not dead.

Table 1: Knowledge base in predicate logic format

Next we should eliminate all implication rules and convert everything into Conjunctive Normal Form (CNF).

#	Predicate Logic
1	$\neg food(X) \vee likes(sarvar, X)$
2	$food(apple)$
3	$food(pear)$
4	$\neg eats(X, Y) \vee dead(X) \vee food(Y)$
5	$eats(alisher, peanuts)$
6	$alive(alisher)$
7	$\neg eats(alisher, X) \vee eats(jamol, X)$
8	$\neg alive(X) \vee \neg dead(X)$

Table 2: Knowledge base in CNF format

Now let's say that we want to prove "Sarvar likes peanuts" using our knowledge base given in Table 2. So we should write our conjecture in predicate format:

#	Predicate Logic	English
*	$likes(sarvar, peanuts)$	Sarvar likes peanuts.

Table 3: Our conjecture to prove

We will use resolution method to prove our above conjecture. For that, we should introduce a *negated conjecture* into our knowledge base and reach a contradiction.

#	Predicate Logic	English
9	$\neg likes(sarvar, peanuts)$	Sarvar does not like peanuts.

Table 4: Our negated conjecture to refute

Let's start deriving new *resolvents* by eliminating complementing terms in our clauses given in Tables 2 and 4.

#	Predicate Logic	Clauses Used
10	$\neg food(peanuts)$	9, 1
11	$\neg eats(X, peanuts) \vee dead(X)$	10, 4
12	$dead(alisher)$	11, 5
13	$\neg alive(alisher)$	12, 8
14	$\emptyset$	13, 6

Table 5: Resolvents

We reached a contradiction, a falsehood similar to $\neg A \wedge A$ form, hence our negated conjecture does not hang together with our knowledge base. Therefore, we proved "Sarvar likes peanuts". $\square$

If you find the tabular form of the proof difficult to trace, we can also draw it in a tree-like form as shown in Figure 1.

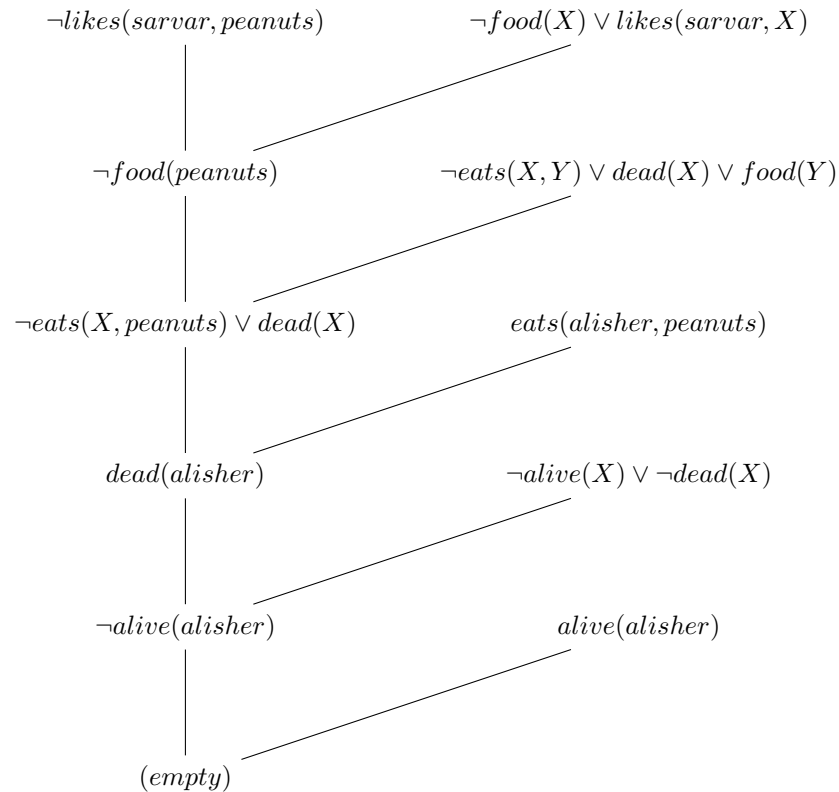


Figure 1: Resolution Tree

2 Programming in Prolog

You should install SWI Prolog on your computer and use Sublime Text 3 to write your Prolog database files. Then you should load your database file into Prolog console and complete tasks requested below.

1. Consider facts and predicates provided in below Prolog database. Save them to a file named `family.pl` and load it to your Prolog console.

```
male(james1).
male(charles1).
male(charles2).
male(james2).
male(george1).

female(catherine).
female(elizabeth).
female(sophia).

parent(charles1, james1).
parent(elizabeth, james1).
parent(charles2, charles1).
parent(catherine, charles1).
parent(james2, charles1).
parent(sophia, elizabeth).
parent(george1, sophia).

mother(X):-
    parent(_,X),
    female(X).

father(X):-
```

```

parent(_,X),
male(X).

sibling(X,Y):-
    parent(X,Z),
    parent(Y,Z),
    X\=Y.

```

2. Query all the fathers from your database and save your answers in a file **fathers.txt**.
3. Query all siblings of **elizabeth** from your database and save your answers in a file **siblings.txt**.
4. Define new rules **brother(X,Y)** and **sister(X,Y)** in your database which can check if **X** has a brother (or a sister) named **Y**. Query all sisters of **charles1** and all brothers of **charles2** and save your results in **charles1_sisters.txt** and **charles2_brothers.txt**.
5. Define new rules **father(X,Y)** and **mother(X,Y)** in your database which can check if **X** is the father (or mother) of **Y**. Query the mother of **charles2** and the father of **elizabeth** and save your results in **charles2_mother.txt** and **elizabeth_father.txt**.
6. Define new rules **grandfather(X,Y)** and **grandmother(X,Y)** in your database which can check if **X** is the grandfather (or grandmother) of **Y**. Find out who has a grandfather or grandmother and write your results in **grandparents.txt**. Hint: you can use **listing** command.
7. Define new rules **nephew(X,Y)** and **niece(X,Y)** in your database which can check if **X** has a nephew (or niece) named as **Y**. Find out who has nephews/nieces and write your results in **nephews.txt**.
8. Define a new rule **ancestor(X,Y)** in your database which can check (recursively) if **X** is an ancestor to **Y**. Find out all the ancestors of **elizabeth** and write your results in **elizabeth_ancestors.txt**.
9. Consider the solution of Tower of Hanoi problem in Prolog given below. Save it as **hanoi.pl**, and load it to your Prolog console. Find out the series of steps for **N=7** problem, and save these steps in **hanoi_solution.txt** file. Compare this solution with your Python solution you completed in previous lab.

```

% move(N,X,Y,Z) - move N disks from peg X to peg Y, with peg Z being the
%                  auxilliary peg
%
% Strategy:
% Base Case: One disc - To transfer a stack consisting of 1 disc from
%   peg X to peg Y, simply move that disc from X to Y
% Recursive Case: To transfer n discs from X to Y, do the following:
%   Transfer the first n-1 discs to some other peg X
%   Move the last disc on X to Y
%   Transfer the n-1 discs from X to peg Y

move(1,X,Y,_):-
    write('Move top disk from '),
    write(X),
    write(' to '),
    write(Y),
    nl.

move(N,X,Y,Z):-
    N>1,
    M is N-1,
    move(M,X,Z,Y),
    move(1,X,Y,_),
    move(M,Z,Y,X).

```

3 Python Tasks

Watch **Week 5: Object-oriented Programming** on this course. Then complete following Python tasks.

1. Implement a class **Vector** in 3D space (i.e. a vector in 3D space). Ensure that any vector can print its coordinates (i.e use (x, y, z) coordinates to determine a point in 3D space), and determine distance between other vector You can use Euclidean distance formula for that:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

You should be able to add or subtract vectors and obtain new vectors too, like this:

$$(x_1, y_1, z_1) + (x_2, y_2, z_2) = (x_1 + x_2, y_1 + y_2, z_1 + z_2)$$

2. Implement a class **Fraction** with numerator and denominator. Ensure that every fraction can print itself, add, subtract, multiple or divide by another fraction. Also implement **reduce** method for converting the fraction into its reduced form. You can use Euclidean algorithm to find GCD for both numerator and denominator, and use this number to reduce your fraction down to its simplest form.
3. Implement a class **IntegerSet** that can contain set of integers. Set should be able to print itself by listing its elements. As any other set in mathematics, it should be able to execute **union**, **difference**, **intersection** and **isMember** operations with other sets (or element).
4. Implement a class **LineSegment** in 2D space which takes 2 coordinates (i.e. start and end of the line). Any line should be able to print its parameters, and also have **isIntersection** and **angle** methods for checking if it intersects with the other line and what the sharp angle between them is.